```
===== src/main.tsx（83 行 / 8 空行） =====
import React from "react";
import ReactDOM from "react-dom/client";
import "./styles/editor.css";
import "./styles/theme-light.css";
import "./styles/theme-dark.css";
const rootEl = document.getElementById("root");
if (!rootEl) {
  console.error("[Clairis] #root element not found!");
}
// Detect which window we're in via URL param
const urlParams = new URLSearchParams(window.location.search);
const isOverlay = urlParams.get("window") === "overlay";
if (isOverlay) {
  // Overlay window: render only the capture overlay
  console.info("[Clairis] Overlay window detected, mounting capture overlay...");
  import("./components/CaptureOverlay").then((mod) => {
    const CaptureOverlay = mod.default;
    ReactDOM.createRoot(rootEl as HTMLElement).render(
      <React.StrictMode>
        <CaptureOverlay />
      </React.StrictMode>,
    );
  });
} else {
  // Main window: render the full app
  console.info("[Clairis] main.tsx loaded, mounting React...");
  window.addEventListener("error", (e) => {
    console.error("[Clairis] Global error:", e.message, e.filename, e.lineno);
  });
  window.addEventListener("unhandledrejection", (e) => {
    console.error("[Clairis] Unhandled promise rejection:", e.reason);
  });
  // Expose main store so capture overlay can call openFile
  import("./stores/index").then((mod) => {
    (window as any).__CLAIRIS_MAIN_STORE__ = mod.useStore;
  });
  // Listen for events from the overlay window
  // Wrapped in .then() + .catch() so browser-only tests don't throw
  import("@tauri-apps/api/event").then(({ listen }) => {
    listen("open-file-in-main", async (event: { payload: { dateStr: string } }) => {
      const { dateStr } = event.payload;
      const toLocalDate = (d: Date) =>
        `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, "0")}-${String(d.getDate()).padStart(2, "0")}`;
      const date =
        dateStr === "今天"
          ? toLocalDate(new Date())
          : dateStr === "昨天"
            ? toLocalDate(new Date(Date.now() - 86400000))
            : dateStr;
```

```
      try {
        const { invoke } = await import("@tauri-apps/api/core");
        const result = await invoke<{ content: string; path: string }>("open_file_direct", {
          path: `~/documents/inbox/inbox-${date}.md`,
        });
        const mainStore = (window as any).__CLAIRIS_MAIN_STORE__;
        if (mainStore && mainStore.getState) {
          mainStore.getState().openFile(result.path, result.content);
        }
      } catch (err) {
        console.warn("[Clairis] open-file-in-main failed:", err);
      }
    }).catch(() => {});
    // Overlay double-click top bar → show capsule in main window
    listen("overlay-to-capsule", () => {
      import("./stores/captureOverlay").then(({ useCaptureOverlayStore }) => {
        useCaptureOverlayStore.getState().minimize();
      });
    }).catch(() => {});
  }).catch(() => {
    // Not in Tauri environment (e.g. browser tests) — skip event listener
  });
  import("./App").then((mod) => {
    const App = mod.default;
    ReactDOM.createRoot(rootEl as HTMLElement).render(
      <React.StrictMode>
        <React.Suspense fallback={<div className="flex items-center justify-center h-screen text-gray-400">Loading...</div
>}>
          <App />
        </React.Suspense>
      </React.StrictMode>,
    );
  });
}
console.info("[Clairis] React render called.");
===== src-tauri/src/main.rs (5 行 / 2 空行) =====
// Prevents additional console window on Windows in release, DO NOT REMOVE!!
#![cfg_attr(not(debug_assertions), windows_subsystem = "windows")]
fn main() {
    clairis_lib::run()
}
===== src-tauri/src/lib.rs (129 行 / 7 空行) =====
mod commands;
mod config;
mod errors;
mod types;
mod validation;
use commands::config_cmd::{load_config, save_config, get_api_key, set_api_key, delete_api_key};
use commands::directory::{read_directory, get_file_info, flatten_files};
use commands::file::{open_file, save_file, save_as_file, open_file_direct, pick_folder};
use commands::translate::{translate_text, stop_translation};
```

```
#[cfg_attr(mobile, tauri::mobile_entry_point)]
pub fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_dialog::init())
        .plugin(tauri_plugin_global_shortcut::Builder::new().build())
        .setup(|app| {
            use tauri::Manager;
            use tauri::webview::WebviewWindowBuilder;
            use tauri::WebviewUrl;
            // Create overlay window (hidden initially, no decorations, always on top)
            let overlay = WebviewWindowBuilder::new(
                app,
                "overlay",
                WebviewUrl::App("index.html?window=overlay".into()),
            )
            .title("")
            .decorations(false)
            .always_on_top(true)
            .inner_size(420.0, 500.0)
            .min_inner_size(320.0, 280.0)
            .visible(false)
            .skip_taskbar(true)
            .build()
            .expect("failed to create overlay window");
            // Set overlay window background transparent (macOS)
            // CRITICAL: setIsOpaque(false) MUST come before setBackgroundColor(.clear)
            // Otherwise macOS ignores the clear background and shows white
            #[cfg(target_os = "macos")]
            {
                use raw_window_handle::HasWindowHandle;
                use std::ptr::null_mut;
                use objc::sel;
                use objc::sel_impl;
                use cocoa::appkit::NSColor;
                use cocoa::base::id;
                if let Ok(handle) = overlay.window_handle() {
                    if let raw_window_handle::RawWindowHandle::AppKit(h) = handle.as_raw() {
                        unsafe {
                            let ns_view = h.ns_view.as_ptr() as id;
                            let ns_window: id = objc::msg_send![ns_view, window];
                            if !ns_window.is_null() {
                                // 1. Tell macOS this window is NOT opaque — required for transparency
                                let _: () = objc::msg_send![ns_window, setIsOpaque: false];
                                // 2. Make titlebar transparent so no gray bar shows
                                let _: () = objc::msg_send![ns_window, setTitlebarAppearsTransparent: true];
                                // 3. Set NSWindow background to clear
                                let clear: id = NSColor::clearColor(null_mut());
                                let _: () = objc::msg_send![ns_window, setBackgroundColor: clear];
                                // 4. Clear WKWebView layer background (the actual white source)
                                let content_view: id = objc::msg_send![ns_window, contentView];
```

```
                            if !content_view.is_null() {
                                let subviews: id = objc::msg_send![content_view, subviews];
                                let count: usize = objc::msg_send![subviews, count];
                                for i in 0..count {
                                    let subview: id = objc::msg_send![subviews, objectAtIndex: i];
                                    let _: () = objc::msg_send![subview, setWantsLayer: true];
                                    let layer: id = objc::msg_send![subview, layer];
                                    if !layer.is_null() {
                                        let _: () = objc::msg_send![layer, setBackgroundColor: clear];
                                    }
                                }
                            }
                        }
                    }
                }
            }
        }
        // Register global shortcut Cmd+Shift+. to toggle overlay
        use tauri_plugin_global_shortcut::{Code, GlobalShortcutExt, Modifiers};
        let handle = app.handle().clone();
        let shortcut = tauri_plugin_global_shortcut::Shortcut::new(
            Some(Modifiers::META | Modifiers::SHIFT),
            Code::Period,
        );
        app.global_shortcut()
            .on_shortcut(shortcut, move |_app, _shortcut, event| {
                if event.state == tauri_plugin_global_shortcut::ShortcutState::Pressed {
                    if let Some(overlay) = handle.get_webview_window("overlay") {
                        if overlay.is_visible().unwrap_or(false) {
                            let _ = overlay.hide();
                        } else {
                            let _ = overlay.show();
                            let _ = overlay.set_focus();
                        }
                    }
                }
            })?;
        Ok(())
    })
    .invoke_handler(tauri::generate_handler![
        open_file,
        open_file_direct,
        pick_folder,
        save_file,
        save_as_file,
        translate_text,
        stop_translation,
        read_directory,
        get_file_info,
        flatten_files,
```

```
            load_config,
            save_config,
            get_api_key,
            set_api_key,
            delete_api_key,
            commands::capture::append_inbox,
            commands::capture::load_today_inbox,
            commands::capture::load_inbox_by_date,
            commands::capture::edit_inbox_entry,
            commands::capture::delete_inbox_entry,
            commands::capture::archive_entry,
            commands::capture::save_config_capture,
            commands::capture::load_config_capture,
            commands::capture::set_overlay_pinned,
            commands::capture::show_overlay_window,
            commands::capture::hide_overlay_window,
        ])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

```
===== src-tauri/src/commands/capture.rs（502 行 / 52 空行） =====
use std::fs;
use serde::{Deserialize, Serialize};
use crate::errors::AppError;
#[derive(Debug, Serialize, Deserialize, Clone)]
#[serde(rename_all = "camelCase")]
pub struct InboxEntry {
    pub id: String,
    pub content: String,
    pub prefix: String,
    pub captured_at: String,
}
#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct InboxEntryInput {
    pub id: String,
    pub content: String,
    pub prefix: String,
    pub captured_at: String,
}
#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct InboxEntryEdit {
    pub date: String,
    pub entry_id: String,
    pub new_content: String,
}
#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct InboxEntryDelete {
```

```
    pub date: String,
    pub entry_id: String,
}
#[derive(Debug, Serialize, Deserialize)]
#[serde(rename_all = "camelCase")]
pub struct InboxEntryArchive {
    pub date: String,
    pub entry_id: String,
}
fn inbox_base_dir() -> Result<std::path::PathBuf, AppError> {
    let home = dirs::home_dir().ok_or(AppError::Io {
        context: "Cannot determine home dir".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, "no home"),
    })?;
    let base = home.join("documents").join("inbox");
    fs::create_dir_all(&base).map_err(|e| AppError::Io {
        context: "create inbox dir".into(),
        source: e,
    })?;
    Ok(base)
}
fn day_file_path(date: &str, active: bool) -> Result<std::path::PathBuf, AppError> {
    let base = inbox_base_dir()?;
    let filename = format!("inbox-{}.md", date);
    if active {
        Ok(base.join(&filename))
    } else {
        fs::create_dir_all(base.join("archive")).map_err(|e| AppError::Io {
            context: "create archive dir".into(),
            source: e,
        })?;
        Ok(base.join("archive").join(&filename))
    }
}
fn format_entry(entry: &InboxEntryInput) -> String {
    let prefix_char = entry.prefix.clone();
    format!(
        "---\ntime: {}\nprefix: {}\n---\n\n{}\n",
        entry.captured_at,
        prefix_char,
        entry.content
    )
}
#[tauri::command]
pub fn append_inbox(entry: InboxEntryInput) -> Result<(), AppError> {
    let date = entry.captured_at.split('T').next().unwrap_or("unknown");
    let path = day_file_path(date, true)?;
    let new_content = format_entry(&entry);
    let mut existing = if path.exists() {
        fs::read_to_string(&path).unwrap_or_default()
```

```
    } else {
        String::new()
    };
    existing.push_str(&new_content);
    let dir = path.parent().unwrap_or_else(|| std::path::Path::new("."));
    let tmp = tempfile::NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp file".into(),
        source: e,
    })?;
    fs::write(&tmp, &existing).map_err(|e| AppError::Io {
        context: "write temp".into(),
        source: e,
    })?;
    tmp.persist(&path).map_err(|e| AppError::Io {
        context: "persist".into(),
        source: e.error,
    })?;
    Ok(())
}
#[tauri::command]
pub fn load_today_inbox() -> Result<Vec<InboxEntry>, AppError> {
    let date = chrono::Local::now().format("%Y-%m-%d").to_string();
    load_inbox_by_date(date)
}
#[tauri::command]
pub fn load_inbox_by_date(date: String) -> Result<Vec<InboxEntry>, AppError> {
    let path = day_file_path(&date, true)?;
    if !path.exists() {
        return Ok(vec![]);
    }
    let content = fs::read_to_string(&path)?;
    let mut entries = Vec::new();
    let blocks: Vec<&str> = content.split("\n---\n").collect();
    for block in &blocks {
        let block = block.trim_start_matches('\n').trim_end();
        if block.is_empty() {
            continue;
        }
        if let Some(end) = block.find("\n\n") {
            let frontmatter = &block[..end];
            let body = &block[end + 2..];
            let mut time = String::new();
            let mut prefix = String::new();
            for line in frontmatter.lines() {
                if let Some(t) = line.strip_prefix("time: ") {
                    time = t.to_string();
                } else if let Some(p) = line.strip_prefix("prefix: ") {
                    prefix = p.to_string();
                }
            }
```

```
            entries.push(InboxEntry {
                id: format!("cap-{}-{}", time.replace(':', "_").replace('.', "_"), body.len()),
                prefix,
                content: body.trim().to_string(),
                captured_at: time,
            });
        }
    }
    Ok(entries.into_iter().rev().collect())
}
#[tauri::command]
pub fn edit_inbox_entry(req: InboxEntryEdit) -> Result<(), AppError> {
    let path = day_file_path(&req.date, true)?;
    if !path.exists() {
        return Err(AppError::Io {
            context: "inbox file not found for edit".into(),
            source: std::io::Error::new(std::io::ErrorKind::NotFound, "file not found"),
        });
    }
    let content = fs::read_to_string(&path)?;
    let time_hint = req.entry_id.rsplit_once('-')
        .map(|(before, _)| before.strip_prefix("cap-").unwrap_or(before).replace('_', "-"))
        .unwrap_or_else(|| req.entry_id.clone());
    let blocks: Vec<&str> = content.split("\n---\n").collect();
    let mut new_content = String::new();
    let mut found = false;
    for (i, block) in blocks.iter().enumerate() {
        let block = block.trim_start_matches('\n').trim_end();
        if block.is_empty() {
            continue;
        }
        if block.contains(&time_hint) && !found {
            // Preserve original prefix
            let mut orig_prefix = String::new();
            if let Some(end) = block.find("\n\n") {
                for line in block[..end].lines() {
                    if let Some(p) = line.strip_prefix("prefix: ") {
                        orig_prefix = p.to_string();
                    }
                }
            }
            if i > 0 && !new_content.is_empty() {
                new_content.push_str("\n---\n");
            }
            new_content.push_str(&format!(
                "---\ntime: {}\nprefix: {}\n---\n\n{}",
                time_hint,
                orig_prefix,
                req.new_content
            ));
```

```
            found = true;
            continue;
        }
        if i > 0 && !new_content.is_empty() {
            new_content.push_str("\n---\n");
        }
        new_content.push_str(block);
    }
    if !found {
        if !new_content.is_empty() {
            new_content.push_str("\n---\n");
        }
        new_content.push_str(&format!(
            "---\ntime: edit-{}\nprefix: \n---\n\n{}",
            chrono::Local::now().to_rfc3339(),
            req.new_content
        ));
    }
    let dir = path.parent().unwrap_or_else(|| std::path::Path::new("."));
    let tmp = tempfile::NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp file for edit".into(),
        source: e,
    })?;
    fs::write(&tmp, &new_content).map_err(|e| AppError::Io {
        context: "write edited content".into(),
        source: e,
    })?;
    tmp.persist(&path).map_err(|e| AppError::Io {
        context: "persist edited file".into(),
        source: e.error,
    })?;
    Ok(())
}
#[tauri::command]
pub fn delete_inbox_entry(req: InboxEntryDelete) -> Result<(), AppError> {
    let path = day_file_path(&req.date, true)?;
    if !path.exists() {
        return Ok(());
    }
    let content = fs::read_to_string(&path)?;
    let time_hint = req.entry_id.rsplit_once('-')
        .map(|(before, _)| before.strip_prefix("cap-").unwrap_or(before).replace('_', "-"))
        .unwrap_or_else(|| req.entry_id.clone());
    let blocks: Vec<&str> = content.split("\n---\n").collect();
    let mut new_content = String::new();
    let mut found = false;
    for (i, block) in blocks.iter().enumerate() {
        let block = block.trim_start_matches('\n').trim_end();
        if block.is_empty() {
            continue;
```

```
        }
        if block.contains(&time_hint) && !found {
            found = true;
            continue;
        }
        if i > 0 && !new_content.is_empty() {
            new_content.push_str("\n---\n");
        }
        new_content.push_str(block);
    }
    let dir = path.parent().unwrap_or_else(|| std::path::Path::new("."));
    let tmp = tempfile::NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp file for delete".into(),
        source: e,
    })?;
    fs::write(&tmp, &new_content).map_err(|e| AppError::Io {
        context: "write after delete".into(),
        source: e,
    })?;
    tmp.persist(&path).map_err(|e| AppError::Io {
        context: "persist deleted file".into(),
        source: e.error,
    })?;
    Ok(())
}
#[tauri::command]
pub fn archive_entry(req: InboxEntryArchive) -> Result<(), AppError> {
    let active_path = day_file_path(&req.date, true)?;
    let archive_path = day_file_path(&req.date, false)?;
    if !active_path.exists() {
        return Ok(());
    }
    let content = fs::read_to_string(&active_path)?;
    let time_hint = req.entry_id.rsplit_once('-')
        .map(|(before, _)| before.strip_prefix("cap-").unwrap_or(before).replace('_', "-"))
        .unwrap_or_else(|| req.entry_id.clone());
    let blocks: Vec<&str> = content.split("\n---\n").collect();
    let mut new_active = String::new();
    let mut archived_text: Option<String> = None;
    for (i, block) in blocks.iter().enumerate() {
        let block = block.trim_start_matches('\n').trim_end();
        if block.is_empty() {
            continue;
        }
        if block.contains(&time_hint) && archived_text.is_none() {
            archived_text = Some(block.to_string());
            continue;
        }
        if i > 0 && !new_active.is_empty() {
            new_active.push_str("\n---\n");
```

```
        }
        new_active.push_str(block);
    }
    let dir = active_path.parent().unwrap_or_else(|| std::path::Path::new("."));
    let tmp = tempfile::NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp file".into(),
        source: e,
    })?;
    fs::write(&tmp, &new_active).map_err(|e| AppError::Io {
        context: "write active after archive".into(),
        source: e,
    })?;
    tmp.persist(&active_path).map_err(|e| AppError::Io {
        context: "persist active".into(),
        source: e.error,
    })?;
    if let Some(entry_text) = archived_text {
        let mut archive_content = if archive_path.exists() {
            fs::read_to_string(&archive_path).unwrap_or_default()
        } else {
            String::new()
        };
        if !archive_content.is_empty() {
            archive_content.push_str("\n---\n");
        }
        archive_content.push_str(&entry_text);
        let archive_dir = archive_path.parent().unwrap_or_else(|| std::path::Path::new("."));
        let tmp_a = tempfile::NamedTempFile::new_in(archive_dir).map_err(|e| AppError::Io {
            context: "create archive temp file".into(),
            source: e,
        })?;
        fs::write(&tmp_a, &archive_content).map_err(|e| AppError::Io {
            context: "write archive".into(),
            source: e,
        })?;
        tmp_a.persist(&archive_path).map_err(|e| AppError::Io {
            context: "persist archive".into(),
            source: e.error,
        })?;
    }
    Ok(())
}
#[derive(Debug, Serialize, Deserialize)]
pub struct CaptureConfig {
    pub font_size: Option<u32>,
    pub position_x: Option<i32>,
    pub position_y: Option<i32>,
    pub capsule_x: Option<f64>,
    pub capsule_y: Option<f64>,
    pub pinned: Option<bool>,
```

```
    pub overlay_width: Option<i32>,
    pub overlay_height: Option<i32>,
    pub panel_ratio: Option<f64>,
}
fn config_path() -> Result<std::path::PathBuf, AppError> {
    let home = dirs::home_dir().ok_or(AppError::Io {
        context: "Cannot determine home dir".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, "no home"),
    })?;
    let dir = home.join(".clairis");
    fs::create_dir_all(&dir).map_err(|e| AppError::Io {
        context: "create .clairis dir".into(),
        source: e,
    })?;
    Ok(dir.join("config.toml"))
}
#[tauri::command]
pub fn save_config_capture(capture_config: CaptureConfig) -> Result<(), AppError> {
    let path = config_path()?;
    let mut doc: toml::Value = if path.exists() {
        let content = fs::read_to_string(&path).unwrap_or_default();
        content.parse::<toml::Value>().unwrap_or(toml::Value::Table(Default::default()))
    } else {
        toml::Value::Table(Default::default())
    };
    let table = doc.as_table_mut().ok_or_else(|| AppError::Io {
        context: "config is not a table".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, "not a table"),
    })?;
    let capture = table.entry("capture".to_string()).or_insert_with(|| {
        toml::Value::Table(Default::default())
    }).as_table_mut().ok_or_else(|| AppError::Io {
        context: "capture section is not a table".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, "not a table"),
    })?;
    if let Some(fs) = capture_config.font_size {
        capture.insert("font_size".to_string(), toml::Value::Integer(fs as i64));
    }
    if let Some(px) = capture_config.position_x {
        capture.insert("position_x".to_string(), toml::Value::Integer(px as i64));
    }
    if let Some(py) = capture_config.position_y {
        capture.insert("position_y".to_string(), toml::Value::Integer(py as i64));
    }
    if let Some(cx) = capture_config.capsule_x {
        capture.insert("capsule_x".to_string(), toml::Value::Float(cx));
    }
    if let Some(cy) = capture_config.capsule_y {
        capture.insert("capsule_y".to_string(), toml::Value::Float(cy));
    }
```

```
    if let Some(p) = capture_config.pinned {
        capture.insert("pinned".to_string(), toml::Value::Boolean(p));
    }
    if let Some(w) = capture_config.overlay_width {
        capture.insert("overlay_width".to_string(), toml::Value::Integer(w as i64));
    }
    if let Some(h) = capture_config.overlay_height {
        capture.insert("overlay_height".to_string(), toml::Value::Integer(h as i64));
    }
    if let Some(r) = capture_config.panel_ratio {
        capture.insert("panel_ratio".to_string(), toml::Value::Float(r));
    }
    let toml_str = toml::to_string_pretty(&doc).map_err(|e| AppError::Io {
        context: "serialize config".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
    })?;
    let dir = path.parent().unwrap_or_else(|| std::path::Path::new("."));
    let tmp = tempfile::NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp config".into(),
        source: e,
    })?;
    fs::write(&tmp, &toml_str).map_err(|e| AppError::Io {
        context: "write config".into(),
        source: e,
    })?;
    tmp.persist(&path).map_err(|e| AppError::Io {
        context: "persist config".into(),
        source: e.error,
    })?;
    Ok(())
}
#[tauri::command]
pub fn set_overlay_pinned(app: tauri::AppHandle, pinned: bool) -> Result<(), AppError> {
    use tauri::Manager;
    if let Some(window) = app.get_webview_window("overlay") {
        window.set_always_on_top(pinned).map_err(|e| AppError::Io {
            context: "set always on top".into(),
            source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
        })?;
    }
    Ok(())
}
#[tauri::command]
pub fn show_overlay_window(app: tauri::AppHandle) -> Result<(), AppError> {
    use tauri::Manager;
    if let Some(window) = app.get_webview_window("overlay") {
        window.show().map_err(|e| AppError::Io {
            context: "show overlay".into(),
            source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
        })?;
```

```
        window.set_focus().map_err(|e| AppError::Io {
            context: "focus overlay".into(),
            source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
        })?;
    }
    Ok(())
}
#[tauri::command]
pub fn hide_overlay_window(app: tauri::AppHandle) -> Result<(), AppError> {
    use tauri::Manager;
    if let Some(window) = app.get_webview_window("overlay") {
        window.hide().map_err(|e| AppError::Io {
            context: "hide overlay".into(),
            source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
        })?;
    }
    Ok(())
}
#[tauri::command]
pub fn load_config_capture() -> Result<CaptureConfig, AppError> {
    let path = config_path()?;
    if !path.exists() {
        return Ok(CaptureConfig {
            font_size: Some(15),
            position_x: None,
            position_y: None,
            capsule_x: None,
            capsule_y: None,
            pinned: None,
            overlay_width: None,
            overlay_height: None,
            panel_ratio: None,
        });
    }
    let content = fs::read_to_string(&path)?;
    let doc: toml::Value = content.parse::<toml::Value>().map_err(|e| AppError::Io {
        context: "parse config.toml".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
    })?;
    let font_size = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("font_size"))
        .and_then(|v: &toml::Value| v.as_integer())
        .map(|v| v as u32);
    let position_x = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("position_x"))
        .and_then(|v: &toml::Value| v.as_integer())
        .map(|v| v as i32);
    let position_y = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("position_y"))
        .and_then(|v: &toml::Value| v.as_integer())
```

```
        .map(|v| v as i32);
    let capsule_x = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("capsule_x"))
        .and_then(|v: &toml::Value| v.as_float());
    let capsule_y = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("capsule_y"))
        .and_then(|v: &toml::Value| v.as_float());
    let pinned = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("pinned"))
        .and_then(|v: &toml::Value| v.as_bool());
    let overlay_width = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("overlay_width"))
        .and_then(|v: &toml::Value| v.as_integer())
        .map(|v| v as i32);
    let overlay_height = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("overlay_height"))
        .and_then(|v: &toml::Value| v.as_integer())
        .map(|v| v as i32);
    let panel_ratio = doc.get("capture")
        .and_then(|c: &toml::Value| c.get("panel_ratio"))
        .and_then(|v: &toml::Value| v.as_float());
    Ok(CaptureConfig { font_size, position_x, position_y, capsule_x, capsule_y, pinned, overlay_width, overlay_height, pane
l_ratio })
}
===== src-tauri/src/commands/config_cmd.rs（46 行 / 9 空行） =====
use tauri::AppHandle;
use crate::config;
use crate::config::AppConfig;
use crate::errors::AppError;
/// Load configuration from disk.
#[tauri::command]
pub async fn load_config(app: AppHandle) -> Result<AppConfig, AppError> {
    config::load_config(&app).await
}
/// Save configuration to disk.
#[tauri::command]
pub async fn save_config(app: AppHandle, config: AppConfig) -> Result<(), AppError> {
    config::save_config(&app, &config).await
}
/// Get API key from system keyring.
#[tauri::command]
pub async fn get_api_key(service: String, account: String) -> Result<Option<String>, AppError> {
    config::get_api_key(&service, &account).await
}
/// Set API key in system keyring.
#[tauri::command]
pub async fn set_api_key(service: String, account: String, key: String) -> Result<(), AppError> {
    config::set_api_key(&service, &account, &key).await
}
/// Delete API key from system keyring.
#[tauri::command]
```

```
pub async fn delete_api_key(service: String, account: String) -> Result<(), AppError> {
    config::delete_api_key(&service, &account).await
}
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn test_config_default() {
        let config = AppConfig::default();
        assert_eq!(config.font_size, Some(16.0));
        assert_eq!(
            config.ollama_endpoint,
            Some("http://localhost:11434".into())
        );
        assert_eq!(
            config.translation_provider,
            Some("ollama".into())
        );
    }
}
===== src-tauri/src/commands/directory.rs (312 行 / 50 空行) =====
use std::fs;
use std::path::PathBuf;
use serde::Serialize;
use tracing::warn;
use crate::errors::AppError;
#[derive(Debug, Serialize)]
pub struct DirEntry {
    pub name: String,
    pub path: String,
    pub is_dir: bool,
    pub size: u64,
}
#[derive(Debug, Serialize)]
pub struct FlatEntry {
    pub name: String,
    pub path: String,
    pub relative_path: String,
    pub is_dir: bool,
    pub size: u64,
}
#[derive(Debug, Serialize)]
pub struct DirectoryResult {
    pub entries: Vec<DirEntry>,
    pub path: String,
}
#[derive(Debug, Serialize)]
pub struct FileInfo {
    pub path: String,
    pub size: u64,
```

```
    pub is_dir: bool,
    pub modified: u64,
}
/// Recursively read a directory (max_depth=3).
/// Excludes .git, node_modules, and hidden files (dotfiles).
#[tauri::command]
pub async fn read_directory(path: String) -> Result<DirectoryResult, AppError> {
    let root = PathBuf::from(&path);
    if !root.is_dir() {
        return Err(AppError::Io {
            context: format!("not a directory: {}", path),
            source: std::io::Error::new(std::io::ErrorKind::NotADirectory, "not a directory"),
        });
    }
    let mut entries = Vec::new();
    collect_entries(&root, &root, 0, 3, &mut entries)?;
    Ok(DirectoryResult {
        entries,
        path,
    })
}
fn collect_entries(
    root: &PathBuf,
    current: &PathBuf,
    depth: usize,
    max_depth: usize,
    entries: &mut Vec<DirEntry>,
) -> Result<(), AppError> {
    if depth > max_depth {
        return Ok(());
    }
    let dir = match fs::read_dir(current) {
        Ok(d) => d,
        Err(e) => {
            warn!(path = %current.display(), error = %e, "Skipping unreadable directory");
            return Ok(());
        }
    };
    for entry in dir.flatten() {
        let path = entry.path();
        // Skip hidden files/directories (dotfiles)
        if let Some(name) = path.file_name().and_then(|n| n.to_str()) {
            if name.starts_with('.') {
                continue;
            }
        }
        // Skip symlinks
        if path.is_symlink() {
            continue;
        }
```

```
        let metadata = match fs::metadata(&path) {
            Ok(m) => m,
            Err(_) => continue,
        };
        let relative = path
            .strip_prefix(root)
            .unwrap_or(&path)
            .to_string_lossy()
            .into_owned();
        entries.push(DirEntry {
            name: path
                .file_name()
                .unwrap_or_default()
                .to_string_lossy()
                .into_owned(),
            path: relative,
            is_dir: metadata.is_dir(),
            size: metadata.len(),
        });
        if metadata.is_dir() {
            collect_entries(root, &path, depth + 1, max_depth, entries)?;
        }
    }
    Ok(())
}
/// Get file metadata info.
#[tauri::command]
pub async fn get_file_info(path: String) -> Result<FileInfo, AppError> {
    let p = PathBuf::from(&path);
    let metadata = fs::metadata(&p).map_err(|e| AppError::Io {
        context: format!("get info for {}", path),
        source: e,
    })?;
    let modified = metadata
        .modified()
        .ok()
        .and_then(|t| t.duration_since(std::time::UNIX_EPOCH).ok())
        .map(|d| d.as_secs())
        .unwrap_or(0);
    Ok(FileInfo {
        path,
        size: metadata.len(),
        is_dir: metadata.is_dir(),
        modified,
    })
}
/// Heuristic heavy directories to skip during recursive scan.
fn is_heavy_dir(name: &str) -> bool {
    matches!(
        name,
```

```
        "node_modules"
            | ".git"
            | "target"
            | "dist"
            | ".next"
            | "__pycache__"
            | "vendor"
            | ".cache"
    )
}
/// Recursively list supported files as a flat list with relative paths.
/// Runs on blocking thread pool to avoid freezing the UI.
/// Max 2000 files, depth 5, skips heavy dirs.
#[tauri::command]
pub async fn flatten_files(path: String) -> Result<FlattenResult, AppError> {
    let base = PathBuf::from(&path);
    if !base.is_dir() {
        return Err(AppError::Io {
            context: "not a directory".into(),
            source: std::io::Error::new(std::io::ErrorKind::NotADirectory, path),
        });
    }
    tokio::task::spawn_blocking(move || {
        let supported = |name: &str| -> bool {
            name.ends_with(".md")
                || name.ends_with(".markdown")
                || name.ends_with(".html")
                || name.ends_with(".htm")
                || name.ends_with(".txt")
        };
        let mut entries: Vec<FlatEntry> = Vec::new();
        let mut truncated = false;
        collect_flat(
            &base, &base, 0, 5, &supported, &mut entries,
            &mut truncated,
        );
        entries.sort_by(|a, b| a.relative_path.cmp(&b.relative_path));
        Ok(FlattenResult {
            entries,
            truncated,
        })
    })
    .await
    .map_err(|e| AppError::Io {
        context: "flatten_files join error".into(),
        source: std::io::Error::new(std::io::ErrorKind::Other, e.to_string()),
    })?
}
#[derive(Debug, Serialize)]
pub struct FlattenResult {
```

```
    pub entries: Vec<FlatEntry>,
    pub truncated: bool,
}
const MAX_FILES: usize = 2000;
fn collect_flat(
    base: &PathBuf,
    current: &PathBuf,
    depth: u32,
    max_depth: u32,
    supported: &dyn Fn(&str) -> bool,
    entries: &mut Vec<FlatEntry>,
    truncated: &mut bool,
) {
    if depth > max_depth || *truncated {
        return;
    }
    let dir = match fs::read_dir(current) {
        Ok(d) => d,
        Err(_) => return,
    };
    for entry in dir.flatten() {
        if *truncated {
            break;
        }
        let path = entry.path();
        let name = path
            .file_name()
            .map(|n| n.to_string_lossy().to_string())
            .unwrap_or_default();
        // Skip hidden
        if name.starts_with('.') {
            continue;
        }
        let metadata = match entry.metadata() {
            Ok(m) => m,
            Err(_) => continue,
        };
        let relative_path = path
            .strip_prefix(base)
            .unwrap_or(&path)
            .to_string_lossy()
            .to_string();
        if metadata.is_dir() {
            if is_heavy_dir(&name) {
                continue;
            }
            entries.push(FlatEntry {
                name: name.clone(),
                path: path.to_string_lossy().to_string(),
                relative_path,
```

```
                is_dir: true,
                size: 0,
            });
            collect_flat(
                base, &path, depth + 1, max_depth,
                supported, entries, truncated,
            );
        } else if supported(&name) {
            if entries.len() >= MAX_FILES {
                *truncated = true;
                break;
            }
            entries.push(FlatEntry {
                name,
                path: path.to_string_lossy().to_string(),
                relative_path,
                is_dir: false,
                size: metadata.len(),
            });
        }
    }
}
#[cfg(test)]
mod tests {
    use super::*;
    use std::fs;
    fn setup_test_dir(label: &str) -> PathBuf {
        let dir = std::env::temp_dir().join(format!("clairis_test_{}_{}", label, std::process::id()));
        let _ = fs::remove_dir_all(&dir);
        fs::create_dir_all(dir.join("subdir")).unwrap();
        fs::write(dir.join("file1.md"), "# Hello").unwrap();
        fs::write(dir.join("subdir").join("file2.md"), "## Nested").unwrap();
        // Hidden file (should be excluded)
        fs::write(dir.join(".hidden"), "secret").unwrap();
        dir
    }
    #[test]
    fn test_read_directory_basic() {
        let dir = setup_test_dir("basic");
        let mut entries = Vec::new();
        let result = collect_entries(&dir, &dir, 0, 3, &mut entries);
        assert!(result.is_ok());
        let names: Vec<&str> = entries.iter().map(|e| e.name.as_str()).collect();
        assert!(names.contains(&"file1.md"), "should contain file1.md: {:?}", names);
        assert!(names.contains(&"subdir"), "should contain subdir: {:?}", names);
        assert!(!names.contains(&".hidden"), "should not contain hidden files: {:?}", names);
        let _ = fs::remove_dir_all(&dir);
    }
    #[test]
    fn test_read_directory_max_depth() {
```

```
        let dir = setup_test_dir("maxdepth");
        let mut entries = Vec::new();
        let result = collect_entries(&dir, &dir, 0, 0, &mut entries);
        assert!(result.is_ok());
        let names: Vec<&str> = entries.iter().map(|e| e.name.as_str()).collect();
        assert!(names.contains(&"file1.md"), "should contain file1.md at depth 0");
        // subdir should appear but its children should not
        assert!(names.contains(&"subdir"), "should contain subdir");
        assert!(!names.contains(&"file2.md"), "should NOT contain file2.md beyond max_depth");
        let _ = fs::remove_dir_all(&dir);
    }
    #[test]
    fn test_read_directory_nonexistent() {
        let mut entries = Vec::new();
        let result = collect_entries(
            &PathBuf::from("/nonexistent_path_xyz"),
            &PathBuf::from("/nonexistent_path_xyz"),
            0,
            3,
            &mut entries,
        );
        assert!(result.is_ok());
        assert!(entries.is_empty());
    }
    #[test]
    fn test_get_file_info() {
        let dir = setup_test_dir("fileinfo");
        let file_path = dir.join("file1.md");
        let metadata = fs::metadata(&file_path).unwrap();
        assert!(metadata.len() > 0);
        let _ = fs::remove_dir_all(&dir);
    }
}
===== src-tauri/src/commands/file.rs (130 行 / 26 空行) =====
use std::fs;
use std::path::PathBuf;
use serde::Serialize;
use tauri::AppHandle;
use tauri_plugin_dialog::DialogExt;
use tempfile::NamedTempFile;
use tracing::info;
use crate::errors::AppError;
#[derive(Serialize)]
pub struct FileResult {
    pub content: String,
    pub path: String,
    pub file_size: u64,
}
#[derive(Serialize)]
pub struct SaveResult {
```

```
    pub path: String,
}
const SOURCE_READ_ONLY_LIMIT: u64 = 2 * 1024 * 1024; // 2MB
const SOURCE_MODE_LIMIT: u64 = 500 * 1024; // 500KB
/// Open a file via system dialog and read its contents.
#[tauri::command]
pub async fn open_file(app: AppHandle) -> Result<FileResult, AppError> {
    let file = app
        .dialog()
        .file()
        .add_filter("文档", &["md", "markdown", "mdown", "mdwn", "html", "htm", "txt"])
        .blocking_pick_file();
    let path = match file {
        Some(p) => p.into_path().unwrap(),
        None => return Err(AppError::Cancelled),
    };
    let content = fs::read_to_string(&path).map_err(|e| AppError::Io {
        context: "read file".into(),
        source: e,
    })?;
    let file_size = content.len() as u64;
    info!(path = %path.display(), size = file_size, "File opened");
    Ok(FileResult {
        content,
        path: path.to_string_lossy().into_owned(),
        file_size,
    })
}
/// Open a file by direct path (used for drag-and-drop).
/// Supports `~` expansion for home directory.
#[tauri::command]
pub async fn open_file_direct(path: String) -> Result<FileResult, AppError> {
    let expanded = if path.starts_with("~/") || path.starts_with("~\\") {
        if let Some(home) = dirs::home_dir() {
            home.join(&path[2..]).to_string_lossy().to_string()
        } else {
            path.clone()
        }
    } else {
        path.clone()
    };
    let path_buf = PathBuf::from(&expanded);
    let content = fs::read_to_string(&path_buf).map_err(|e| AppError::Io {
        context: "read file".into(),
        source: e,
    })?;
    let file_size = content.len() as u64;
    info!(path = %expanded, size = file_size, "File opened via direct path");
    Ok(FileResult {
        content,
```

```
        path: expanded,
        file_size,
    })
}
/// Save file to the given path atomically.
#[tauri::command]
pub async fn save_file(path: String, content: String) -> Result<SaveResult, AppError> {
    let target = PathBuf::from(&path);
    // Ensure parent directory exists
    if let Some(parent) = target.parent() {
        fs::create_dir_all(parent).map_err(|e| AppError::Io {
            context: "create parent dir".into(),
            source: e,
        })?;
    }
    // Atomic write: write to temp file, then persist (rename)
    let dir = target
        .parent()
        .unwrap_or_else(|| std::path::Path::new("."));
    let tmp = NamedTempFile::new_in(dir).map_err(|e| AppError::Io {
        context: "create temp file".into(),
        source: e,
    })?;
    fs::write(&tmp, &content).map_err(|e| AppError::Io {
        context: "write temp file".into(),
        source: e,
    })?;
    tmp.persist(&target).map_err(|e| AppError::Io {
        context: "persist temp file".into(),
        source: e.error,
    })?;
    info!(path = %path, "File saved atomically");
    Ok(SaveResult { path })
}
/// Open "Save As" dialog and write file.
#[tauri::command]
pub async fn save_as_file(app: AppHandle, content: String) -> Result<SaveResult, AppError> {
    let file = app
        .dialog()
        .file()
        .add_filter("文档", &["md", "markdown", "html", "htm", "txt"])
        .blocking_save_file();
    let path = match file {
        Some(p) => p.into_path().unwrap(),
        None => return Err(AppError::Cancelled),
    };
    let path_str = path.to_string_lossy().into_owned();
    save_file(path_str.clone(), content).await?;
    Ok(SaveResult { path: path_str })
}
```

```
/// Open a folder picker dialog and return the selected path.
#[tauri::command]
pub async fn pick_folder(app: AppHandle) -> Result<String, AppError> {
    let folder = app.dialog().file().blocking_pick_folder();
    match folder {
        Some(p) => Ok(p.into_path().unwrap().to_string_lossy().into_owned()),
        None => Err(AppError::Cancelled),
    }
}
/// Return the size thresholds so the frontend can decide editor mode.
#[tauri::command]
pub fn get_size_limits() -> (u64, u64) {
    (SOURCE_MODE_LIMIT, SOURCE_READ_ONLY_LIMIT)
}
===== src-tauri/src/commands/mod.rs（5 行 / 1 空行） =====
pub mod capture;
pub mod config_cmd;
pub mod directory;
pub mod file;
pub mod translate;
===== src-tauri/src/commands/translate.rs（227 行 / 23 空行） =====
use futures_util::StreamExt;
use reqwest::Client;
use serde::Deserialize;
use std::sync::atomic::{AtomicBool, Ordering};
use tauri::{AppHandle, Emitter};
use tracing::info;
use crate::config::load_config;
use crate::errors::AppError;
use crate::types::TranslationChunk;
static CANCELLED: AtomicBool = AtomicBool::new(false);
#[derive(Deserialize)]
pub struct TranslateRequest {
    pub text: String,
    pub source_lang: String,
    pub target_lang: String,
    pub provider: String,
}
async fn translate_ollama(
    client: &Client,
    endpoint: &str,
    prompt: &str,
    request_id: &str,
    app: &AppHandle,
) -> Result<(), AppError> {
    let url = format!("{}/api/chat", endpoint.trim_end_matches('/'));
    let body = serde_json::json!({
        "model": "llama3",
        "stream": true,
        "messages": [{"role": "user", "content": prompt}]
```

```
    });
    let mut stream = client
        .post(&url)
        .json(&body)
        .send()
        .await
        .map_err(|e: reqwest::Error| AppError::Translation {
            context: "request".into(),
            source: e.to_string(),
        })?
        .bytes_stream();
    while let Some(chunk) = stream.next().await {
        if CANCELLED.load(Ordering::SeqCst) {
            return Ok(());
        }
        let chunk = chunk.map_err(|e: reqwest::Error| AppError::Translation {
            context: "stream".into(),
            source: e.to_string(),
        })?;
        let raw = String::from_utf8_lossy(&chunk);
        for line in raw.lines() {
            if line.trim().is_empty() {
                continue;
            }
            if let Ok(ollama_resp) = serde_json::from_str::<OllamaChunk>(line) {
                if let Some(content) = ollama_resp.message.and_then(|m| Some(m.content)) {
                    let _ =
                        app.emit("translation-chunk", TranslationChunk {
                            request_id: request_id.to_string(),
                            text: content,
                            done: false,
                            error: None,
                        });
                }
                if ollama_resp.done {
                    let _ =
                        app.emit("translation-chunk", TranslationChunk {
                            request_id: request_id.to_string(),
                            text: String::new(),
                            done: true,
                            error: None,
                        });
                }
            }
        }
    }
    Ok(())
}
async fn translate_openai(
    client: &Client,
```

```
    endpoint: &str,
    api_key: &str,
    model: &str,
    prompt: &str,
    request_id: &str,
    app: &AppHandle,
) -> Result<(), AppError> {
    let url = format!("{}/v1/chat/completions", endpoint.trim_end_matches('/'));
    let body = serde_json::json!({
        "model": model,
        "stream": true,
        "messages": [{"role": "user", "content": prompt}]
    });
    let mut stream = client
        .post(&url)
        .header("Authorization", format!("Bearer {}", api_key))
        .json(&body)
        .send()
        .await
        .map_err(|e: reqwest::Error| AppError::Translation {
            context: "request".into(),
            source: e.to_string(),
        })?
        .bytes_stream();
    let mut buffer = String::new();
    while let Some(chunk) = stream.next().await {
        if CANCELLED.load(Ordering::SeqCst) {
            return Ok(());
        }
        let chunk = chunk.map_err(|e: reqwest::Error| AppError::Translation {
            context: "stream".into(),
            source: e.to_string(),
        })?;
        buffer.push_str(&String::from_utf8_lossy(&chunk));
        while let Some(line_end) = buffer.find('\n') {
            let line = buffer[..line_end].trim().to_string();
            buffer.drain(..=line_end);
            if line.is_empty() || line.starts_with(":") {
                continue;
            }
            if line == "data: [DONE]" {
                let _ = app.emit("translation-chunk", TranslationChunk {
                    request_id: request_id.to_string(),
                    text: String::new(),
                    done: true,
                    error: None,
                });
                return Ok(());
            }
            if let Some(data) = line.strip_prefix("data: ") {
```

```
                if let Ok(openai_chunk) = serde_json::from_str::<OpenAIChunk>(data) {
                    if let Some(delta) = openai_chunk.choices.first() {
                        if let Some(content) = &delta.delta.content {
                            let _ = app.emit("translation-chunk", TranslationChunk {
                                request_id: request_id.to_string(),
                                text: content.clone(),
                                done: false,
                                error: None,
                            });
                        }
                    }
                }
            }
        }
    }
    Ok(())
}
#[tauri::command]
pub async fn translate_text(
    app: AppHandle,
    request: TranslateRequest,
) -> Result<(), AppError> {
    let request_id = uuid_v4();
    info!(%request_id, provider = %request.provider, "Translation started");
    // Abort previous request
    CANCELLED.store(true, Ordering::SeqCst);
    tokio::time::sleep(tokio::time::Duration::from_millis(50)).await;
    CANCELLED.store(false, Ordering::SeqCst);
    let config = load_config(&app).await.unwrap_or_default();
    let client = Client::new();
    let prompt = format!(
        "Translate the following text from {} to {}. Return ONLY the translation, no explanations.\n\n{}",
        request.source_lang, request.target_lang, request.text
    );
    match request.provider.as_str() {
        "ollama" => {
            translate_ollama(
                &client,
                &config.ollama_endpoint.unwrap_or_default(),
                &prompt,
                &request_id,
                &app,
            )
            .await
        }
        "openai" => {
            translate_openai(
                &client,
                &config.openai_endpoint.unwrap_or_default(),
                &config.openai_api_key.unwrap_or_default(),
```

```
                &config.openai_model.unwrap_or_default(),
                &prompt,
                &request_id,
                &app,
            )
            .await
        }
        _ => Err(AppError::Translation {
            context: "provider".into(),
            source: format!("Unknown provider: {}", request.provider),
        }),
    }
}
/// Cancel the current translation request.
#[tauri::command]
pub async fn stop_translation() -> Result<(), AppError> {
    CANCELLED.store(true, std::sync::atomic::Ordering::SeqCst);
    Ok(())
}
fn uuid_v4() -> String {
    use std::time::{SystemTime, UNIX_EPOCH};
    let nanos = SystemTime::now()
        .duration_since(UNIX_EPOCH)
        .unwrap_or_default()
        .as_nanos();
    format!("req-{:016x}", nanos)
}
#[derive(Deserialize)]
struct OllamaChunk {
    message: Option<OllamaMessage>,
    done: bool,
}
#[derive(Deserialize)]
struct OllamaMessage {
    content: String,
}
#[derive(Deserialize)]
struct OpenAIChunk {
    choices: Vec<OpenAIChoice>,
}
#[derive(Deserialize)]
struct OpenAIChoice {
    delta: OpenAIDelta,
}
#[derive(Deserialize)]
struct OpenAIDelta {
    content: Option<String>,
}
===== src-tauri/src/config.rs（153 行 / 13 空行） =====
use serde::{Deserialize, Serialize};
```

```
use std::path::PathBuf;
use tauri::{AppHandle, Manager};
use tracing::{info, warn};
use crate::errors::AppError;
#[derive(Debug, Serialize, Deserialize, Clone)]
#[serde(rename_all = "camelCase")]
pub struct AppConfig {
    pub font_size: Option<f64>,
    pub heading_colors: Option<HeadingColors>,
    pub ollama_endpoint: Option<String>,
    pub openai_endpoint: Option<String>,
    pub openai_model: Option<String>,
    #[serde(skip)]
    pub openai_api_key: Option<String>,
    pub translation_provider: Option<String>,
    pub source_lang: Option<String>,
    pub target_lang: Option<String>,
    pub theme: Option<String>,
    pub sidebar_left_visible: Option<bool>,
    pub sidebar_right_visible: Option<bool>,
    pub sidebar_left_width: Option<f64>,
    pub sidebar_right_width: Option<f64>,
    pub horizontal_split_ratio: Option<f64>,
    pub vertical_split_ratio: Option<f64>,
    pub recent_files: Option<Vec<RecentFile>>,
}
#[derive(Debug, Serialize, Deserialize, Clone)]
pub struct RecentFile {
    pub path: String,
    pub title: String,
    pub opened_at: String,
}
impl Default for AppConfig {
    fn default() -> Self {
        Self {
            font_size: Some(16.0),
            heading_colors: None,
            ollama_endpoint: Some("http://localhost:11434".into()),
            openai_endpoint: Some("https://api.openai.com".into()),
            openai_model: Some("gpt-4o-mini".into()),
            openai_api_key: None,
            translation_provider: Some("ollama".into()),
            source_lang: Some("en".into()),
            target_lang: Some("zh-CN".into()),
            theme: Some("system".into()),
            sidebar_left_visible: Some(true),
            sidebar_right_visible: Some(false),
            sidebar_left_width: Some(220.0),
            sidebar_right_width: Some(320.0),
            horizontal_split_ratio: Some(0.5),
```

```
          return resolved !== url ? `url('${resolved}')` : `url('${url}')`;
        },
      );
      return `${open}${rewritten}${close}`;
    },
  );
  return result;
}
/**
 * Resolve a potentially-relative URL against a base directory.
 * Returns the original URL if it's absolute/external or can't be resolved.
 */
function resolvePath(url: string, baseDir: string): string {
  const trimmed = url.trim();
  // Skip absolute URLs, data URIs, anchor-only, and already-converted asset URLs
  if (
    !trimmed ||
    trimmed.startsWith("http://") ||
    trimmed.startsWith("https://") ||
    trimmed.startsWith("data:") ||
    trimmed.startsWith("file://") ||
    trimmed.startsWith("#") ||
    trimmed.startsWith("asset://") ||
    trimmed.startsWith("tauri://")
  ) {
    return url;
  }
  // Resolve relative path against base directory
  const resolved = resolveRelativePath(trimmed, baseDir);
  if (!resolved) return url;
  // Convert to Tauri asset URL
  try {
    return convertFileSrc(resolved);
  } catch {
    return url;
  }
}
/**
 * Get the parent directory path from a file path.
 */
function getDirPath(filePath: string): string | null {
  const lastSep = Math.max(filePath.lastIndexOf("/"), filePath.lastIndexOf("\\"));
  if (lastSep === -1) return null;
  return filePath.substring(0, lastSep);
}
/**
 * Resolve a relative path against a base directory.
 */
function resolveRelativePath(relative: string, baseDir: string): string | null {
  // Normalize separators
```

```
  const norm = relative.replace(/\\/g, "/");
  const sep = "/";
  // Already absolute (starts with /) — treat as relative to filesystem root
  // (this is rare for local HTML, but handle gracefully)
  if (norm.startsWith("/")) {
    return norm;
  }
  // Combine base directory with relative path
  const parts = [...baseDir.split(sep).filter(Boolean)];
  for (const part of norm.split(sep)) {
    if (part === "." || part === "") continue;
    if (part === "..") {
      parts.pop();
    } else {
      parts.push(part);
    }
  }
  if (parts.length === 0) return null;
  return `/${parts.join(sep)}`;
}
===== src/renderers/index.ts (21 行 / 3 空行) =====
import type { FileKind } from "../types";
import { renderMarkdown, processMarkdownLinks } from "./markdown";
import { renderText } from "./text";
export { renderMarkdown, processMarkdownLinks } from "./markdown";
export { rewriteHtmlPaths } from "./html";
export { renderText } from "./text";
/**
 * Render document content to HTML based on its file kind.
 * Returns processed HTML that PreviewView can render directly
 * (via dangerouslySetInnerHTML for safe types, or pass to HtmlPreview for html).
 */
export function renderByKind(kind: FileKind, content: string): string {
  switch (kind) {
    case "markdown":
      return processMarkdownLinks(renderMarkdown(content));
    case "html":
      return content; // Raw — HtmlPreview handles srcdoc + path rewriting
    case "text":
      return renderText(content);
  }
}
===== src/renderers/markdown.ts (28 行 / 4 空行) =====
import { marked } from "marked";
import DOMPurify from "dompurify";
export function renderMarkdown(content: string): string {
  if (!content) return "";
  const raw = marked.parse(content, { async: false }) as string;
  return DOMPurify.sanitize(raw, {
    ALLOWED_TAGS: [
```

```
      "p", "h1", "h2", "h3", "h4", "h5", "h6",
      "ul", "ol", "li", "pre", "code",
      "blockquote", "table", "thead", "tbody", "tr", "th", "td",
      "img", "a", "em", "strong", "del", "hr", "br",
    ],
    ALLOWED_ATTR: ["href", "src", "alt", "title", "class", "target"],
    ALLOW_DATA_ATTR: false,
    ADD_ATTR: ["target"],
  });
}
/** Add target="_blank" to all links after sanitization */
export function processMarkdownLinks(html: string): string {
  const parser = new DOMParser();
  const doc = parser.parseFromString(html, "text/html");
  doc.querySelectorAll("a").forEach((a) => {
    if (!a.getAttribute("target")) {
      a.setAttribute("target", "_blank");
    }
  });
  return doc.body.innerHTML;
}
===== src/renderers/text.ts（11 行 / 1 空行） =====
/**
 * Render plain text as HTML-safe <pre> content.
 */
export function renderText(content: string): string {
  if (!content) return "";
  const escaped = content
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;");
  return `<pre style="white-space:pre-wrap;word-wrap:break-word">${escaped}</pre>`;
}
===== src/stores/captureOverlay.ts（370 行 / 36 空行） =====
import { create } from 'zustand';
import type { CaptureStatus, CapturePrefix, InboxEntry, UndoState } from '../types';
let _idCounter = 0;
function nextEntryId(): string {
  _idCounter += 1;
  const ts = new Date().toISOString().replace(/[:.]/g, '_');
  return `cap-${ts}-${_idCounter}`;
}
const UNDO_MS = 5000;
let _fontSaveTimer: ReturnType<typeof setTimeout> | null = null;
const IME_PREFIX_MAP: Record<string, CapturePrefix> = {
  '.': '.', '。': '.', '､': '.',
  '-': '-', '－': '-', '—': '-', '–': '-',
  '!': '!', '！': '!',
  '?': '?', '？': '?',
};
```

```
export interface CaptureOverlayStore {
  status: CaptureStatus;
  draftContent: string;
  draftPrefix: CapturePrefix;
  history: InboxEntry[];
  displayLimit: number;
  collapsed: boolean;
  selectedIndex: number;
  positionX: number;
  positionY: number;
  undo: UndoState | null;
  savedAckId: string | null;
  fontSize: number;
  editingId: string | null;
  editBuffer: string;
  capsuleX: number;
  capsuleY: number;
  pinned: boolean;
  overlayWidth: number;
  overlayHeight: number;
  panelRatio: number;
  open: () => void;
  toggle: () => void;
  close: () => void;
  minimize: () => void;
  restore: () => void;
  // Primary draft setter — handles per-line prefix detection
  setDraftContent: (raw: string, cursorPos?: number) => void;
  // Alias for compatibility
  setDraftInput: (raw: string) => void;
  // Direct prefix setter (used by Enter empty-line clear)
  setDraftPrefix: (prefix: CapturePrefix) => void;
  handleCmdEnter: () => Promise<string | null>;
  handleCmdN: () => Promise<void>;
  archiveEntry: (id: string) => void;
  undoArchive: (id: string) => void;
  tickUndo: () => void;
  selectNext: () => void;
  selectPrev: () => void;
  setSelectedIndex: (i: number) => void;
  setCollapsed: (v: boolean) => void;
  loadMore: () => void;
  setPosition: (x: number, y: number) => void;
  setFontSize: (s: number) => void;
  clampPosition: (overlayW?: number, overlayH?: number) => void;
  loadHistory: () => Promise<void>;
  startEdit: (id: string, content: string) => void;
  setEditBuffer: (v: string) => void;
  saveEdit: () => Promise<void>;
  cancelEdit: () => void;
```

```
  deleteEntry: (id: string) => void;
  openInMainWindow: (dateStr: string) => void;
  loadConfig: () => Promise<void>;
  setCapsulePosition: (x: number, y: number) => void;
  clampCapsulePosition: () => void;
  setPinned: (v: boolean) => void;
  setOverlaySize: (w: number, h: number) => void;
  setPanelRatio: (r: number) => void;
}
export const useCaptureOverlayStore = create<CaptureOverlayStore>((set, get) => ({
  status: 'closed',
  draftContent: '',
  draftPrefix: '',
  history: [],
  displayLimit: 5,
  collapsed: true,
  selectedIndex: -1,
  positionX: -1,
  positionY: 60,
  undo: null,
  savedAckId: null,
  fontSize: 15,
  editingId: null,
  editBuffer: '',
  capsuleX: 0,
  capsuleY: 0,
  pinned: false,
  overlayWidth: 420,
  overlayHeight: 500,
  panelRatio: 0.4,
  open: () => {
    const { positionX, overlayWidth } = get();
    // On first open (positionX === -1), position at right-top
    if (positionX === -1) {
      const x = Math.max(20, window.innerWidth - overlayWidth - 24);
      set({ status: 'normal', selectedIndex: -1, positionX: x });
    } else {
      set({ status: 'normal', selectedIndex: -1 });
    }
    get().loadHistory();
    get().loadConfig();
  },
  toggle: () => {
    const { status, positionX, overlayWidth } = get();
    if (status === 'closed' || status === 'capsule') {
      const patch: Partial<CaptureOverlayStore> = { status: 'normal', selectedIndex: -1 };
      if (positionX === -1) {
        patch.positionX = Math.max(20, window.innerWidth - overlayWidth - 24);
      }
      set(patch);
```

```
      get().loadHistory();
    } else {
      set({ status: 'closed' });
    }
  },
  close: () => set({ status: 'closed', draftContent: '', draftPrefix: '', selectedIndex: -1, editingId: null, editBuffer:
'' }),
  minimize: () => set({ status: 'capsule' }),
  restore: () => set({ status: 'normal', selectedIndex: -1 }),
  setDraftContent: (raw: string, cursorPos?: number) => {
    const pos = cursorPos ?? raw.length;
    // Per-line prefix detection: find the line containing the cursor
    const beforeCursor = raw.slice(0, pos);
    const lastNewline = beforeCursor.lastIndexOf('\n');
    const currentLineStart = lastNewline + 1;
    const posInLine = pos - currentLineStart;
    // If cursor is at position 1 in the current line (first char just typed)
    if (posInLine === 1 && raw.length > 0) {
      const firstChar = raw[currentLineStart];
      const detected = IME_PREFIX_MAP[firstChar];
      if (detected) {
        // Consume the prefix char: strip it from content
        const newContent = raw.slice(0, currentLineStart) + raw.slice(currentLineStart + 1);
        set({ draftPrefix: detected, draftContent: newContent });
        return;
      }
    }
    // Default: just update content
    set({ draftContent: raw });
  },
  setDraftPrefix: (prefix: CapturePrefix) => set({ draftPrefix: prefix }),
  setDraftInput: (raw: string, cursorPos?: number) => get().setDraftContent(raw, cursorPos),
  handleCmdEnter: async (): Promise<string | null> => {
    const { draftContent, draftPrefix, undo } = get();
    if (undo) return null;
    if (!draftContent.trim() && draftPrefix) {
      set({ draftPrefix: '', draftContent: '' });
      return null;
    }
    if (!draftContent.trim()) return null;
    const id = nextEntryId();
    const capturedAt = new Date().toISOString();
    const entry: InboxEntry = { id, prefix: draftPrefix, content: draftContent.trim(), capturedAt };
    try {
      const { invoke } = await import('@tauri-apps/api/core');
      await invoke('append_inbox', { entry: { id, content: entry.content, prefix: entry.prefix, capturedAt } });
    } catch {
      const fallback = JSON.parse(localStorage.getItem('clairis_capture_fallback') || '[]');
      fallback.push(entry);
      localStorage.setItem('clairis_capture_fallback', JSON.stringify(fallback));
    }
```

```
    // Clear prefix after submission
    set(s => ({ history: [entry, ...s.history], draftContent: '', draftPrefix: '', selectedIndex: -1, savedAckId: id }));
    setTimeout(() => set(s => (s.savedAckId === id ? { savedAckId: null } : {})), 600);
    // Restore textarea focus
    requestAnimationFrame(() => {
      const ta = document.querySelector('textarea[data-capture-input]') as HTMLTextAreaElement | null;
      if (ta) ta.focus();
    });
    return id;
  },
  handleCmdN: async () => {
    const { draftContent } = get();
    if (draftContent.trim()) await get().handleCmdEnter();
    // If no content, just clear content (keep prefix for sticky mode)
  },
  archiveEntry: (id: string) => {
    const deadline = Date.now() + UNDO_MS;
    const timer = window.setTimeout(() => get().tickUndo(), UNDO_MS + 50);
    set(s => ({
      undo: { entryId: id, deadline, timer },
      history: s.history.map(e => e.id === id ? { ...e, archivedAt: new Date().toISOString() } : e),
    }));
  },
  undoArchive: (id: string) => {
    const { undo } = get();
    if (undo?.timer) window.clearTimeout(undo.timer);
    set(s => ({
      undo: null,
      history: s.history.map(e => e.id === id ? { ...e, archivedAt: undefined } : e),
    }));
  },
  tickUndo: () => {
    const { undo, history } = get();
    if (!undo) return;
    if (Date.now() < undo.deadline) {
      const msLeft = undo.deadline - Date.now();
      const timer = window.setTimeout(() => get().tickUndo(), msLeft + 50);
      set(s => (s.undo ? { undo: { ...s.undo, timer } } : {}));
      return;
    }
    const entry = history.find(e => e.id === undo.entryId);
    set(s => ({ undo: null, history: s.history.filter(e => e.id !== undo.entryId) }));
    if (entry) {
      import('@tauri-apps/api/core').then(({ invoke }) => {
        invoke('archive_entry', { date: entry.capturedAt.slice(0, 10), entryId: entry.id }).catch(() => {});
      });
    }
  },
  selectNext: () => set(s => ({ selectedIndex: Math.min(s.selectedIndex + 1, s.history.length - 1) })),
  selectPrev: () => set(s => ({ selectedIndex: Math.max(s.selectedIndex - 1, -1) })),
```

```
  setSelectedIndex: (i: number) => set({ selectedIndex: i }),
  setCollapsed: (v: boolean) => set({ collapsed: v }),
  loadMore: () => set(s => ({ displayLimit: s.displayLimit + 20 })),
  setPosition: (x: number, y: number) => set({ positionX: x, positionY: y }),
  clampPosition: (overlayW?: number, overlayH?: number) => {
    const padding = 20;
    const w = overlayW ?? 420;
    const h = overlayH ?? 420;
    const maxX = window.innerWidth - w - padding;
    const maxY = window.innerHeight - h - padding;
    const { positionX, positionY } = get();
    const newX = Math.max(padding, Math.min(positionX, maxX));
    const newY = Math.max(padding, Math.min(positionY, maxY));
    set({ positionX: newX, positionY: newY });
    // Persist position
    import('@tauri-apps/api/core').then(({ invoke }) => {
      invoke('save_config_capture', {
        captureConfig: { position_x: Math.round(newX), position_y: Math.round(newY) }
      }).catch(() => {});
    });
  },
  setFontSize: (s: number) => {
    const clamped = Math.max(12, Math.min(22, s));
    set({ fontSize: clamped });
    if (_fontSaveTimer) clearTimeout(_fontSaveTimer);
    _fontSaveTimer = setTimeout(() => {
      import('@tauri-apps/api/core').then(({ invoke }) => {
        invoke('save_config_capture', { captureConfig: { fontSize: clamped } });
      });
    }, 500);
  },
  loadHistory: async () => {
    try {
      const { invoke } = await import('@tauri-apps/api/core');
      const today = await invoke<InboxEntry[]>('load_today_inbox');
      const toLocalDate = (d: Date) => `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate
()).padStart(2, '0')}`;
      const yesterday = toLocalDate(new Date(Date.now() - 86400000));
      const yesterdayEntries = await invoke<InboxEntry[]>('load_inbox_by_date', { date: yesterday });
      const all = [...today, ...yesterdayEntries];
      const seen = new Set<string>();
      const unique = all.filter(e => { if (seen.has(e.id)) return false; seen.add(e.id); return true; });
      set({ history: unique });
    } catch {
      const fallback: InboxEntry[] = JSON.parse(localStorage.getItem('clairis_capture_fallback') || '[]');
      set({ history: fallback });
    }
  },
  startEdit: (id: string, content: string) => set({ editingId: id, editBuffer: content }),
  setEditBuffer: (v: string) => set({ editBuffer: v }),
  saveEdit: async () => {
```

```
    const { editingId, editBuffer, history } = get();
    if (!editingId) return;
    const entry = history.find(e => e.id === editingId);
    if (!entry) return;
    const newContent = editBuffer.trim();
    if (!newContent) return;
    try {
      const { invoke } = await import('@tauri-apps/api/core');
      await invoke('edit_inbox_entry', { req: { date: entry.capturedAt.slice(0, 10), entryId: editingId, newContent } });
    } catch {
      const fallback: InboxEntry[] = JSON.parse(localStorage.getItem('clairis_capture_fallback') || '[]');
      const updated = fallback.map(e => e.id === editingId ? { ...e, content: newContent } : e);
      localStorage.setItem('clairis_capture_fallback', JSON.stringify(updated));
    }
    set(s => ({
      history: s.history.map(e => e.id === editingId ? { ...e, content: newContent } : e),
      editingId: null, editBuffer: '',
    }));
  },
  cancelEdit: () => set({ editingId: null, editBuffer: '' }),
  deleteEntry: (id: string) => {
    const entry = get().history.find(e => e.id === id);
    set(s => ({ history: s.history.filter(e => e.id !== id) }));
    if (entry) {
      import('@tauri-apps/api/core').then(({ invoke }) => {
        invoke('delete_inbox_entry', { date: entry.capturedAt.slice(0, 10), entryId: id }).catch(() => {});
      });
    }
  },
  openInMainWindow: (dateStr: string) => {
    const toLocalDate = (d: Date) => `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate
()).padStart(2, '0')}`;
    const date = dateStr === '今天'
      ? toLocalDate(new Date())
      : dateStr === '昨天'
        ? toLocalDate(new Date(Date.now() - 86400000))
        : dateStr;
    // Overlay window: always emit event to main window (overlay can't access main store)
    import('@tauri-apps/api/event').then(({ emit }) => {
      emit('open-file-in-main', { dateStr: date });
    }).catch(() => {});
  },
  loadConfig: async () => {
    try {
      const { invoke } = await import('@tauri-apps/api/core');
      const config = await invoke<{ fontSize?: number; positionX?: number; positionY?: number; capsuleX?: number; capsule
Y?: number; pinned?: boolean; overlayWidth?: number; overlayHeight?: number; panelRatio?: number }>('load_config_capture');
      const patch: Partial<CaptureOverlayStore> = {};
      if (config.fontSize !== undefined && config.fontSize !== null) patch.fontSize = config.fontSize;
      if (config.positionX !== undefined && config.positionX !== null) patch.positionX = config.positionX;
      if (config.positionY !== undefined && config.positionY !== null) patch.positionY = config.positionY;
      if (config.capsuleX !== undefined && config.capsuleX !== null) patch.capsuleX = config.capsuleX;
```

```
      if (config.capsuleY !== undefined && config.capsuleY !== null) patch.capsuleY = config.capsuleY;
      if (config.pinned !== undefined && config.pinned !== null) patch.pinned = config.pinned;
      if (config.overlayWidth !== undefined && config.overlayWidth !== null) patch.overlayWidth = config.overlayWidth;
      if (config.overlayHeight !== undefined && config.overlayHeight !== null) patch.overlayHeight = config.overlayHeight;
      if (config.panelRatio !== undefined && config.panelRatio !== null) patch.panelRatio = config.panelRatio;
      if (Object.keys(patch).length > 0) set(patch);
    } catch {
      // config not found or parse error — use defaults
    }
  },
  setCapsulePosition: (x: number, y: number) => set({ capsuleX: x, capsuleY: y }),
  clampCapsulePosition: () => {
    const padding = 20;
    const size = 44;
    const maxX = window.innerWidth - size - padding;
    const maxY = window.innerHeight - size - padding;
    const { capsuleX, capsuleY } = get();
    const newX = Math.max(padding, Math.min(capsuleX, maxX));
    const newY = Math.max(padding, Math.min(capsuleY, maxY));
    set({ capsuleX: newX, capsuleY: newY });
    // Persist capsule position
    import('@tauri-apps/api/core').then(({ invoke }) => {
      invoke('save_config_capture', {
        captureConfig: { capsule_x: newX, capsule_y: newY }
      }).catch(() => {});
    });
  },
  setPinned: (v: boolean) => {
    set({ pinned: v });
    import('@tauri-apps/api/core').then(({ invoke }) => {
      invoke('set_overlay_pinned', { pinned: v }).catch(() => {});
    });
  },
  setOverlaySize: (w: number, h: number) => {
    const clampedW = Math.max(320, Math.min(600, Math.round(w)));
    const clampedH = Math.max(280, Math.min(800, Math.round(h)));
    set({ overlayWidth: clampedW, overlayHeight: clampedH });
    get().clampPosition(clampedW, clampedH);
    import('@tauri-apps/api/core').then(({ invoke }) => {
      invoke('save_config_capture', {
        captureConfig: { overlay_width: clampedW, overlay_height: clampedH }
      }).catch(() => {});
    });
  },
  setPanelRatio: (r: number) => {
    const clamped = Math.max(0.2, Math.min(0.8, Math.round(r * 100) / 100));
    set({ panelRatio: clamped });
    import('@tauri-apps/api/core').then(({ invoke }) => {
      invoke('save_config_capture', {
        captureConfig: { panel_ratio: clamped }
```

```
      }).catch(() => {});
    });
  },
}));
===== src/stores/editorStore.ts (111 行 / 27 空行) =====
// @deprecated — 考虑合并到 useStore (src/stores/index.ts)。当前两个 store 分离
// 导致跨 store 操作复杂 (config 在 editorStore, tabs 在 useStore)。
// Phase 1a 后评估合并。
import { create } from "zustand";
import type {
  AppConfig,
  EditorMode,
  HeadingColors,
  TranslationState,
  RawMarkdown,
} from "../types";
export interface EditorStore {
  // File state
  filePath: string | null;
  fileName: string | null;
  fileSize: number | null;
  content: string;
  // Editor mode
  editorMode: EditorMode;
  // Font size
  fontSize: number;
  // Heading colors
  headingColors: HeadingColors;
  // Translation
  translation: TranslationState;
  // Dirty flag
  dirty: boolean;
  // Version counter — incremented on external content changes (file open, mode switch)
  // WYSIWYG editor uses this to know when to push content into TipTap
  contentVersion: number;
  // Config
  config: Partial<AppConfig>;
  // Config loading/saving
  configLoaded: boolean;
  configLoading: boolean;
  // Actions
  setContent: (content: string) => void;
  incContentVersion: () => void;
  setFilePath: (path: string | null, name: string | null) => void;
  setFileSize: (size: number | null) => void;
  setEditorMode: (mode: EditorMode) => void;
  setFontSize: (size: number) => void;
  setHeadingColors: (colors: HeadingColors) => void;
  setTranslation: (state: Partial<TranslationState>) => void;
  setDirty: (dirty: boolean) => void;
```

```
  setConfig: (config: Partial<AppConfig>) => void;
  setConfigLoaded: (loaded: boolean) => void;
  setConfigLoading: (loading: boolean) => void;
  reset: () => void;
}
const DEFAULT_HEADING_COLORS: HeadingColors = {
  h1: "#d73a49",
  h2: "#6f42c1",
  h3: "#005cc5",
  h4: "#e36209",
  h5: "#24292e",
  h6: "#586069",
};
const initialState = {
  filePath: null as string | null,
  fileName: null as string | null,
  fileSize: null as number | null,
  content: "",
  editorMode: "preview" as EditorMode,
  fontSize: 16,
  headingColors: DEFAULT_HEADING_COLORS,
  translation: {
    active: false,
    requestId: null as string | null,
    text: "",
    loading: false,
    error: null as string | null,
  },
  dirty: false,
  contentVersion: 0,
  config: {},
  configLoaded: false,
  configLoading: false,
};
export const useEditorStore = create<EditorStore>((set) => ({
  ...initialState,
  setContent: (content) =>
    set({ content, dirty: true }),
  incContentVersion: () =>
    set((prev) => ({ contentVersion: prev.contentVersion + 1 })),
  setFilePath: (path, name) =>
    set({ filePath: path, fileName: name, dirty: false }),
  setFileSize: (size) =>
    set({ fileSize: size }),
  setEditorMode: (mode) =>
    set({ editorMode: mode }),
  setFontSize: (size) =>
    set({ fontSize: size }),
  setHeadingColors: (colors) =>
    set({ headingColors: colors }),
```

```
  setTranslation: (state) =>
    set((prev) => ({
      translation: { ...prev.translation, ...state },
    })),
  setDirty: (dirty) =>
    set({ dirty }),
  setConfig: (config) =>
    set((prev) => ({
      config: { ...prev.config, ...config },
    })),
  setConfigLoaded: (loaded) =>
    set({ configLoaded: loaded }),
  setConfigLoading: (loading) =>
    set({ configLoading: loading }),
  reset: () => set(initialState),
}));
===== src/stores/index.ts (280 行 / 43 空行) =====
import { create } from "zustand";
import type { Tab, EditorMode, FileKind, RecentFile } from "../types";
export type SplitDirection = "horizontal" | "vertical";
let nextId = 1;
function generateId(): string {
  return `tab-${nextId++}`;
}
function detectFileKind(path: string): FileKind {
  if (path.endsWith(".md") || path.endsWith(".markdown")) return "markdown";
  if (path.endsWith(".html") || path.endsWith(".htm")) return "html";
  return "text";
}
function extractFileName(path: string): string {
  const parts = path.split("/");
  return parts[parts.length - 1] || path;
}
interface AppStore {
  activeTabId: string | null;
  tabs: Record<string, Tab>;
  sidebarLeftVisible: boolean;
  sidebarRightVisible: boolean;
  readingMode: boolean;
  recentFiles: RecentFile[];
  // Split panel — each panel holds its own tab refs
  splitDirection: SplitDirection | null;
  splitTabIds: string[];
  splitActiveTabId: string | null;
  splitEditorMode: EditorMode;
  horizontalSplitRatio: number;
  verticalSplitRatio: number;
  reset: () => void;
  openFile: (path: string, content: string) => void;
  newTab: () => void;
```

```
  switchTab: (id: string) => void;
  closeTab: (id: string) => void;
  updateContent: (content: string) => void;
  updateTabContent: (tabId: string, content: string) => void;
  setEditorMode: (mode: EditorMode) => void;
  setReadingMode: (on: boolean) => void;
  markClean: () => void;
  toggleLeftSidebar: () => void;
  toggleRightSidebar: () => void;
  setSidebarLeftVisible: (v: boolean) => void;
  setSidebarRightVisible: (v: boolean) => void;
  addRecentFile: (path: string, title: string) => void;
  clearRecentFiles: () => void;
  // Split actions
  splitTab: (tabId: string, direction: SplitDirection) => void;
  unsplit: () => void;
  moveTabToSplit: (tabId: string) => void;
  moveTabToMain: (tabId: string) => void;
  setSplitEditorMode: (mode: EditorMode) => void;
  setSplitActiveTab: (id: string) => void;
  setSplitRatio: (direction: SplitDirection, ratio: number) => void;
  setSplitRatios: (ratios: { horizontal?: number; vertical?: number }) => void;
}
const initialState = {
  activeTabId: null as string | null,
  tabs: {} as Record<string, Tab>,
  sidebarLeftVisible: true,
  sidebarRightVisible: false,
  readingMode: false,
  recentFiles: [] as RecentFile[],
  splitDirection: null as SplitDirection | null,
  splitTabIds: [] as string[],
  splitActiveTabId: null as string | null,
  splitEditorMode: "preview" as EditorMode,
  horizontalSplitRatio: 0.5,
  verticalSplitRatio: 0.5,
};
export const useStore = create<AppStore>((set, get) => ({
  ...initialState,
  reset: () => set(initialState),
  openFile: (path: string, content: string) => {
    const state = get();
    // Check if tab for this path already exists
    const existing = Object.values(state.tabs).find((t) => t.path === path);
    if (existing) {
      set({
        tabs: {
          ...state.tabs,
          [existing.id]: {
            ...existing,
```

```
            content,
            dirty: false,
          },
        },
        activeTabId: existing.id,
      });
      return;
    }
    const fileKind = detectFileKind(path);
    const title = extractFileName(path);
    const id = generateId();
    const tab: Tab = {
      id,
      title,
      path,
      fileKind,
      content,
      dirty: false,
      editorMode: "preview",
      scrollPos: 0,
    };
    set({
      tabs: { ...state.tabs, [id]: tab },
      activeTabId: id,
    });
  },
  switchTab: (id: string) => {
    set({ activeTabId: id });
  },
  closeTab: (id: string) => {
    const state = get();
    const { [id]: _removed, ...remaining } = state.tabs;
    let activeTabId = state.activeTabId;
    if (activeTabId === id) {
      const remainingIds = Object.keys(remaining);
      activeTabId = remainingIds.length > 0 ? remainingIds[0] : null;
    }
    set({ tabs: remaining, activeTabId });
  },
  updateContent: (content: string) => {
    const state = get();
    const { activeTabId, tabs } = state;
    if (!activeTabId) return;
    const tab = tabs[activeTabId];
    if (!tab) return;
    set({
      tabs: {
        ...tabs,
        [activeTabId]: { ...tab, content, dirty: true },
      },
```

```
    });
  },
  updateTabContent: (tabId: string, content: string) => {
    const { tabs } = get();
    const tab = tabs[tabId];
    if (!tab) return;
    set({
      tabs: {
        ...tabs,
        [tabId]: { ...tab, content, dirty: true },
      },
    });
  },
  setEditorMode: (mode: EditorMode) => {
    const state = get();
    const { activeTabId, tabs } = state;
    if (!activeTabId) return;
    const tab = tabs[activeTabId];
    if (!tab) return;
    set({
      tabs: {
        ...tabs,
        [activeTabId]: { ...tab, editorMode: mode },
      },
    });
  },
  setReadingMode: (on: boolean) => {
    set({ readingMode: on });
  },
  markClean: () => {
    const state = get();
    const { activeTabId, tabs } = state;
    if (!activeTabId) return;
    const tab = tabs[activeTabId];
    if (!tab) return;
    set({
      tabs: {
        ...tabs,
        [activeTabId]: { ...tab, dirty: false },
      },
    });
  },
  newTab: () => {
    const id = generateId();
    const tab: Tab = {
      id,
      title: "未命名",
      path: null,
      fileKind: "markdown",
      content: "",
```

```
      dirty: false,
      editorMode: "edit",
      scrollPos: 0,
    };
    set((s) => ({
      tabs: { ...s.tabs, [id]: tab },
      activeTabId: id,
    }));
  },
  toggleLeftSidebar: () =>
    set((s) => ({ sidebarLeftVisible: !s.sidebarLeftVisible })),
  toggleRightSidebar: () =>
    set((s) => ({ sidebarRightVisible: !s.sidebarRightVisible })),
  setSidebarLeftVisible: (v: boolean) => set({ sidebarLeftVisible: v }),
  setSidebarRightVisible: (v: boolean) => set({ sidebarRightVisible: v }),
  addRecentFile: (path: string, title: string) => {
    set((s) => {
      const filtered = s.recentFiles.filter((f) => f.path !== path);
      const entry: RecentFile = { path, title, openedAt: new Date().toISOString() };
      return { recentFiles: [entry, ...filtered].slice(0, 10) };
    });
  },
  clearRecentFiles: () => set({ recentFiles: [] }),
  // ── Split panel actions ──
  splitTab: (tabId: string, direction: SplitDirection) =>
    set((s) => {
      // Move the tab to the split panel
      const newSplitIds = [tabId];
      // Keep any existing split tabs that are still valid
      for (const id of s.splitTabIds) {
        if (id !== tabId && s.tabs[id] && !newSplitIds.includes(id)) {
          newSplitIds.push(id);
        }
      }
      return {
        splitDirection: direction,
        splitTabIds: newSplitIds,
        splitActiveTabId: tabId,
        // If the split tab was the main active tab, switch main to another tab
        activeTabId: s.activeTabId === tabId
          ? Object.keys(s.tabs).find((id) => id !== tabId && !newSplitIds.includes(id)) || null
          : s.activeTabId,
      };
    }),
  unsplit: () =>
    set({
      splitDirection: null,
      splitTabIds: [],
      splitActiveTabId: null,
    }),
```

```
  moveTabToSplit: (tabId: string) =>
    set((s) => {
      if (s.splitTabIds.includes(tabId)) return {};
      return {
        splitTabIds: [...s.splitTabIds, tabId],
        splitActiveTabId: tabId,
        activeTabId: s.activeTabId === tabId
          ? Object.keys(s.tabs).find((id) => id !== tabId && !s.splitTabIds.includes(id)) || null
          : s.activeTabId,
      };
    }),
  moveTabToMain: (tabId: string) =>
    set((s) => ({
      splitTabIds: s.splitTabIds.filter((id) => id !== tabId),
      splitActiveTabId: s.splitActiveTabId === tabId
        ? s.splitTabIds.find((id) => id !== tabId) || null
        : s.splitActiveTabId,
      activeTabId: tabId,
    })),
  setSplitEditorMode: (mode: EditorMode) => set({ splitEditorMode: mode }),
  setSplitActiveTab: (id: string) => set({ splitActiveTabId: id }),
  setSplitRatio: (direction: SplitDirection, ratio: number) => {
    const clamped = Math.max(0.1, Math.min(0.9, ratio));
    if (direction === "horizontal") {
      set({ horizontalSplitRatio: clamped });
    } else {
      set({ verticalSplitRatio: clamped });
    }
  },
  setSplitRatios: (ratios) => {
    const patch: Partial<Pick<AppStore, "horizontalSplitRatio" | "verticalSplitRatio">> = {};
    if (ratios.horizontal !== undefined) {
      patch.horizontalSplitRatio = Math.max(0.1, Math.min(0.9, ratios.horizontal));
    }
    if (ratios.vertical !== undefined) {
      patch.verticalSplitRatio = Math.max(0.1, Math.min(0.9, ratios.vertical));
    }
    set(patch as Partial<AppStore>);
  },
}));
// Expose store for E2E testing (Tauri Mock bridge)
if (typeof window !== "undefined") {
  (window as unknown as Record<string, unknown>).__clairis_store__ = useStore;
}
// Re-export capture overlay store
export { useCaptureOverlayStore } from './captureOverlay';
export type { CaptureOverlayStore } from './captureOverlay';
===== src/styles/editor.css（244 行 / 54 空行） =====
@tailwind base;
@tailwind components;
```

```
@tailwind utilities;
:root {
  --editor-font-size: 16px;
}
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
html,
body,
#root {
  height: 100%;
  width: 100%;
  overflow: hidden;
}
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", "PingFang SC",
    "Hiragino Sans GB", "Microsoft YaHei", "Helvetica Neue", Helvetica, Arial,
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
/* Editor area */
.editor-area {
  font-size: var(--editor-font-size);
  line-height: 1.7;
}
/* ── CodeMirror ──────────────────────────────────────── */
.cm-editor {
  height: 100%;
  font-size: var(--editor-font-size);
  color: #1f2937;
  background: #ffffff;
}
.cm-editor .cm-scroller {
  font-family: "SF Mono", "Fira Code", "Fira Mono", "Menlo", Consolas, monospace;
}
.cm-editor .cm-content {
  color: #1f2937;
}
.cm-editor .cm-gutters {
  background: #f9fafb;
  color: #6b7280;
  border-right: 1px solid #e5e7eb;
}
.cm-editor .cm-activeLine {
  background: rgba(59, 130, 246, 0.05);
}
.cm-editor .cm-selectionBackground {
```

```
  background: rgba(59, 130, 246, 0.2);
}
.cm-editor .cm-cursor {
  border-left-color: #1f2937;
}
/* ── Dark Mode: CM6 Editor ──────────────────────────────── */
.dark .cm-editor {
  color: #e5e7eb;
  background: #111827;
}
.dark .cm-editor .cm-content {
  color: #e5e7eb;
}
.dark .cm-editor .cm-gutters {
  background: #1f2937;
  color: #9ca3af;
  border-right-color: #374151;
}
.dark .cm-editor .cm-activeLine {
  background: rgba(59, 130, 246, 0.08);
}
.dark .cm-editor .cm-selectionBackground {
  background: rgba(59, 130, 246, 0.3);
}
.dark .cm-editor .cm-cursor {
  border-left-color: #e5e7eb;
}
/* Editor container mount point */
.cm-editor-container {
  height: 100%;
}
/* ── L2 Decoration Styles ──────────────────────────────── */
/* Bold (**text**) — delimiter hidden, text bold */
.cm-strong {
  font-weight: 700;
}
/* Italic (*text*) — delimiter hidden, text italic */
.cm-em {
  font-style: italic;
}
/* Inline code (`code`) — backticks hidden, monospace + gray bg */
.cm-inline-code {
  font-family: "SF Mono", "Fira Code", "Fira Mono", "Menlo", Consolas, monospace;
  font-size: 0.9em;
  background: rgba(0, 0, 0, 0.06);
  border-radius: 3px;
  padding: 0.15em 0.35em;
}
.dark .cm-inline-code {
  background: rgba(255, 255, 255, 0.1);
```

```
}
/* Heading text — larger, bolder */
.cm-heading {
  font-weight: 700;
  font-size: 1.1em;
}
/* Heading marker widget [H1]-[H6] */
.cm-heading-marker {
  display: inline-block;
  font-size: 0.7em;
  font-weight: 600;
  color: #fff;
  background: #6b7280;
  border-radius: 3px;
  padding: 1px 5px;
  margin-right: 4px;
  vertical-align: middle;
  cursor: pointer;
  user-select: none;
}
.cm-heading-marker:hover {
  background: #4b5563;
}
.dark .cm-heading-marker {
  background: #4b5563;
}
.dark .cm-heading-marker:hover {
  background: #6b7280;
}
/* Inline link — URL hidden, blue clickable text */
.cm-link {
  color: #2563eb;
  text-decoration: underline;
  text-decoration-style: dotted;
  cursor: pointer;
}
.cm-link:hover {
  color: #1d4ed8;
}
.dark .cm-link {
  color: #60a5fa;
}
.dark .cm-link:hover {
  color: #93bbfd;
}
/* List items — marker replaced, no special text styling needed */
.cm-list {
  /* marker is hidden via replace, line inherits normal text style */
}
/* Task items */
```

```
.cm-task-done {
  text-decoration: line-through;
  color: #9ca3af;
}
.dark .cm-task-done {
  color: #6b7280;
}
.cm-task-pending {
  color: inherit;
}
/* Blockquote — > hidden, left border + gray bg applied */
.cm-quote {
  border-left: 3px solid #d1d5db;
  background: rgba(0, 0, 0, 0.02);
  color: #6b7280;
  font-style: italic;
}
.dark .cm-quote {
  border-left-color: #4b5563;
  background: rgba(255, 255, 255, 0.03);
  color: #9ca3af;
}
/* ── SplitResizeHandle ──────────────────────────────────────── */
.split-resize-handle {
  background: transparent;
  transition: background 0.15s;
  user-select: none;
  /* Minimum 16px touch/click target via invisible padding */
  position: relative;
}
.split-resize-handle::before {
  content: "";
  position: absolute;
  inset: -6px;
  /* Expands hit area to ≥ 16px while visual bar stays 4px */
}
/* Horizontal split: vertical bar */
.split-handle-h {
  width: 4px;
  cursor: col-resize;
}
/* Vertical split: horizontal bar */
.split-handle-v {
  height: 4px;
  cursor: row-resize;
}
.split-resize-handle:hover,
.split-resize-handle.dragging {
  background: #93c5fd;
}
```

```
.dark .split-resize-handle:hover,
.dark .split-resize-handle.dragging {
  background: #3b82f6;
}
/* Respect reduced-motion preference */
@media (prefers-reduced-motion: reduce) {
  .split-resize-handle {
    transition: none;
  }
}
/* ── Capture Overlay ──────────────────────────────────────── */
@keyframes overlayIn {
  from { opacity: 0; transform: translateY(-6px) scale(0.97); }
  to   { opacity: 1; transform: translateY(0) scale(1); }
}
.overlay-enter {
  animation: overlayIn 0.22s cubic-bezier(0.2, 0.8, 0.2, 1);
}
@keyframes capsuleIn {
  from { opacity: 0; transform: scale(0.5); }
  to   { opacity: 1; transform: scale(1); }
}
.capsule-enter {
  animation: capsuleIn 0.18s cubic-bezier(0.2, 0.8, 0.2, 1);
}
@keyframes archiveSlideOut {
  0%   { transform: translateX(0); opacity: 1; background: rgba(254, 243, 199, 0.4); }
  100% { transform: translateX(100%); opacity: 0; }
}
@keyframes fadeInSlide {
  0%   { opacity: 0; transform: translateY(-4px); }
  100% { opacity: 1; transform: translateY(0); }
}
@media (prefers-reduced-motion: reduce) {
  .overlay-enter,
  .capsule-enter {
    animation: none;
  }
  [style*="fadeInSlide"] {
    animation: none !important;
  }
}
===== src/styles/theme-dark.css (77 行 / 12 空行)  =====
/* Reader (preview) dark theme */
.dark {
  --reader-font-size: 16px;
  --reader-line-height: 1.8;
  --reader-max-width: 680px;
  --reader-bg: #0d1117;
  --reader-text: #c9d1d9;
```

```
  --reader-link: #58a6ff;
  --reader-code-bg: rgba(240, 246, 252, 0.15);
  --reader-code-text: #c9d1d9;
  --reader-blockquote-border: #30363d;
  --reader-blockquote-text: #8b949e;
  --reader-hr: #21262d;
  --reader-table-border: #30363d;
  --reader-table-header-bg: #161b22;
}
/* PreviewView dark mode */
.dark .preview-view {
  font-size: var(--reader-font-size);
  line-height: var(--reader-line-height);
  color: var(--reader-text);
  background: var(--reader-bg);
  word-wrap: break-word;
}
.dark .preview-view a {
  color: var(--reader-link);
}
.dark .preview-view code {
  background: var(--reader-code-bg);
  color: var(--reader-code-text);
  border-radius: 3px;
  padding: 0.15em 0.35em;
  font-size: 0.9em;
  font-family: "SF Mono", "Fira Code", "Fira Mono", "Menlo", Consolas, monospace;
}
.dark .preview-view pre {
  background: #161b22;
  border-radius: 4px;
  padding: 0.75rem 1rem;
  overflow-x: auto;
}
.dark .preview-view pre code {
  background: none;
  padding: 0;
  border-radius: 0;
}
.dark .preview-view blockquote {
  border-left: 3px solid var(--reader-blockquote-border);
  padding-left: 1em;
  margin: 0.5em 0;
  color: var(--reader-blockquote-text);
}
.dark .preview-view hr {
  border: none;
  border-top: 1px solid var(--reader-hr);
  margin: 1em 0;
}
```

```
.dark .preview-view table {
  border-collapse: collapse;
  width: 100%;
  margin: 0.5em 0;
}
.dark .preview-view th,
.dark .preview-view td {
  border: 1px solid var(--reader-table-border);
  padding: 0.4em 0.6em;
  text-align: left;
  vertical-align: top;
}
.dark .preview-view th {
  background: var(--reader-table-header-bg);
  font-weight: 600;
}
.dark .preview-view img {
  max-width: 100%;
  height: auto;
}
===== src/styles/theme-light.css（77 行 / 12 空行） =====
/* Reader (preview) light theme */
:root {
  --reader-font-size: 16px;
  --reader-line-height: 1.8;
  --reader-max-width: 680px;
  --reader-bg: #ffffff;
  --reader-text: #24292e;
  --reader-link: #0366d6;
  --reader-code-bg: rgba(27, 31, 35, 0.05);
  --reader-code-text: #24292e;
  --reader-blockquote-border: #d1d5db;
  --reader-blockquote-text: #6b7280;
  --reader-hr: #e1e4e8;
  --reader-table-border: #d1d5db;
  --reader-table-header-bg: #f6f8fa;
}
/* PreviewView light mode */
.preview-view {
  font-size: var(--reader-font-size);
  line-height: var(--reader-line-height);
  color: var(--reader-text);
  background: var(--reader-bg);
  word-wrap: break-word;
}
.preview-view a {
  color: var(--reader-link);
}
.preview-view code {
  background: var(--reader-code-bg);
```

```
  color: var(--reader-code-text);
  border-radius: 3px;
  padding: 0.15em 0.35em;
  font-size: 0.9em;
  font-family: "SF Mono", "Fira Code", "Fira Mono", "Menlo", Consolas, monospace;
}
.preview-view pre {
  background: #f6f8fa;
  border-radius: 4px;
  padding: 0.75rem 1rem;
  overflow-x: auto;
}
.preview-view pre code {
  background: none;
  padding: 0;
  border-radius: 0;
}
.preview-view blockquote {
  border-left: 3px solid var(--reader-blockquote-border);
  padding-left: 1em;
  margin: 0.5em 0;
  color: var(--reader-blockquote-text);
}
.preview-view hr {
  border: none;
  border-top: 1px solid var(--reader-hr);
  margin: 1em 0;
}
.preview-view table {
  border-collapse: collapse;
  width: 100%;
  margin: 0.5em 0;
}
.preview-view th,
.preview-view td {
  border: 1px solid var(--reader-table-border);
  padding: 0.4em 0.6em;
  text-align: left;
  vertical-align: top;
}
.preview-view th {
  background: var(--reader-table-header-bg);
  font-weight: 600;
}
.preview-view img {
  max-width: 100%;
  height: auto;
}
===== src/types/index.ts（82 行 / 17 空行） =====
export type EditorMode = "preview" | "edit" | "source";
```

```
export type FileKind = "markdown" | "html" | "text";
export type TranslationProvider = "ollama" | "openai";
export type LangPair = {
  source: string;
  target: string;
};
export interface HeadingColors {
  h1: string;
  h2: string;
  h3: string;
  h4: string;
  h5: string;
  h6: string;
}
export interface AppConfig {
  fontSize: number;
  headingColors: HeadingColors;
  ollamaEndpoint: string;
  openaiEndpoint: string;
  openaiModel: string;
  translationProvider: TranslationProvider;
  langPair: LangPair;
  theme: Theme;
  sidebarLeftVisible: boolean;
  sidebarRightVisible: boolean;
  sidebarLeftWidth: number;
  sidebarRightWidth: number;
  horizontalSplitRatio: number;
  verticalSplitRatio: number;
  recentFiles: RecentFile[];
}
export type Theme = "system" | "light" | "dark";
export type CaptureStatus = "closed" | "normal" | "pinned" | "capsule";
export type CapturePrefix = "" | "." | "—" | "!" | "?";
export type EntryType = "problem" | "todo" | "idea" | "wonder" | "note";
export interface InboxEntry {
  id: string;
  type?: EntryType;
  prefix: CapturePrefix;
  content: string;
  capturedAt: string;
  archivedAt?: string;
}
export interface UndoState {
  entryId: string;
  deadline: number;
  timer?: number;
}
export interface CapturePosition {
  x: number;
```

```
  y: number;
}
export interface RecentFile {
  path: string;
  title: string;
  openedAt: string;
}
export interface Tab {
  id: string;
  title: string;
  path: string | null;
  fileKind: FileKind;
  content: string;
  dirty: boolean;
  editorMode: EditorMode;
  scrollPos: number;
}
export type RawMarkdown = string;
export interface TranslationState {
  active: boolean;
  requestId: string | null;
  text: string;
  loading: boolean;
  error: string | null;
}
export interface TranslationChunk {
  requestId: string;
  text: string;
  done: boolean;
  error?: string;
}
===== src/utils/log.ts（88 行 / 14 空行） =====
/**
 * Unified logger for Clairis.
 * In Tauri: logs to Rust tracing backend (via invoke or console).
 * In browser: logs to console with timestamp and level prefix.
 *
 * Usage:
 *   import { log } from "../utils/log";
 *   log.info("FileTree", "Loading folder", { path });
 *   log.warn("Decoration", "Large file detected", { size });
 *   log.error("SaveFile", "Write failed", e);
 */
type LogLevel = "debug" | "info" | "warn" | "error";
const LEVEL_WEIGHT: Record<LogLevel, number> = {
  debug: 0,
  info: 1,
  warn: 2,
  error: 3,
};
```

```
const MIN_LEVEL: LogLevel = "info"; // Change to "debug" for verbose output
interface LogEntry {
  ts: string;
  module: string;
  level: LogLevel;
  message: string;
  data?: unknown;
}
const MAX_BUFFER = 200;
const buffer: LogEntry[] = [];
function shouldLog(level: LogLevel): boolean {
  return LEVEL_WEIGHT[level] >= LEVEL_WEIGHT[MIN_LEVEL];
}
function format(entry: LogEntry): string {
  const base = `[${entry.ts}] [${entry.level.toUpperCase()}] [${entry.module}] ${entry.message}`;
  if (entry.data !== undefined) {
    const extra =
      entry.data instanceof Error
        ? `${entry.data.message}\n${entry.data.stack}`
        : JSON.stringify(entry.data, null, 0);
    return `${base} — ${extra}`;
  }
  return base;
}
function emit(level: LogLevel, module: string, message: string, data?: unknown) {
  if (!shouldLog(level)) return;
  const entry: LogEntry = {
    ts: new Date().toISOString(),
    module,
    level,
    message,
    data,
  };
  // Ring buffer
  buffer.push(entry);
  if (buffer.length > MAX_BUFFER) buffer.shift();
  const line = format(entry);
  switch (level) {
    case "error":
      console.error(line);
      break;
    case "warn":
      console.warn(line);
      break;
    case "info":
      console.info(line);
      break;
    case "debug":
      console.debug(line);
      break;
```

```
  }
}
export const log = {
  debug: (module: string, message: string, data?: unknown) =>
    emit("debug", module, message, data),
  info: (module: string, message: string, data?: unknown) =>
    emit("info", module, message, data),
  warn: (module: string, message: string, data?: unknown) =>
    emit("warn", module, message, data),
  error: (module: string, message: string, data?: unknown) =>
    emit("error", module, message, data),
  /** Get recent log entries for debugging */
  recent: (count = 50) => buffer.slice(-count),
  /** Clear buffer */
  clear: () => buffer.length = 0,
};
// Expose for console debugging
if (typeof window !== "undefined") {
  (window as unknown as Record<string, unknown>).__clairis_log__ = log;
}
===== src/vite-env.d.ts（1 行 / 1 空行） =====
/// <reference types="vite/client" />
```